ITCS442

Relational Database Design

Project Phase 3: Logical Database Design

Group 5: Restaurant Inventory Management System

Submitted by

6688072 Kemjira Nugboon

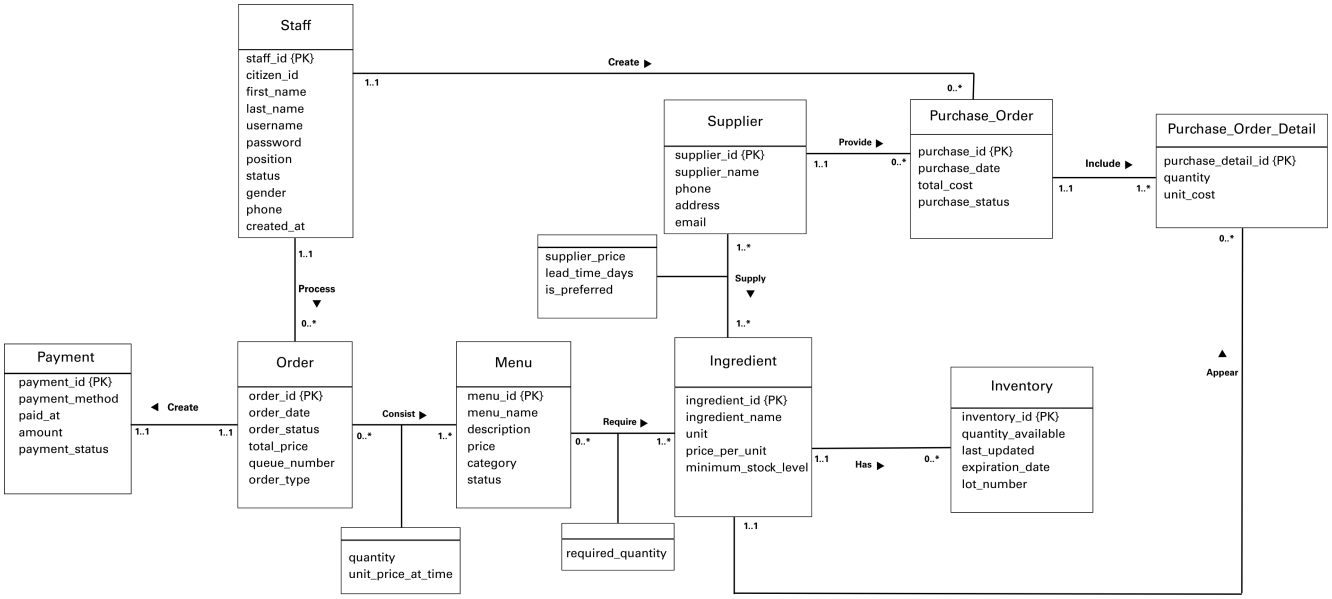
6688099 Nuttanan Reamprasert

6688129 Patsatraporn Thongdeesakul

Presented to

Asst.Prof. Dr. Charnyote Pluempitiwiriyawej

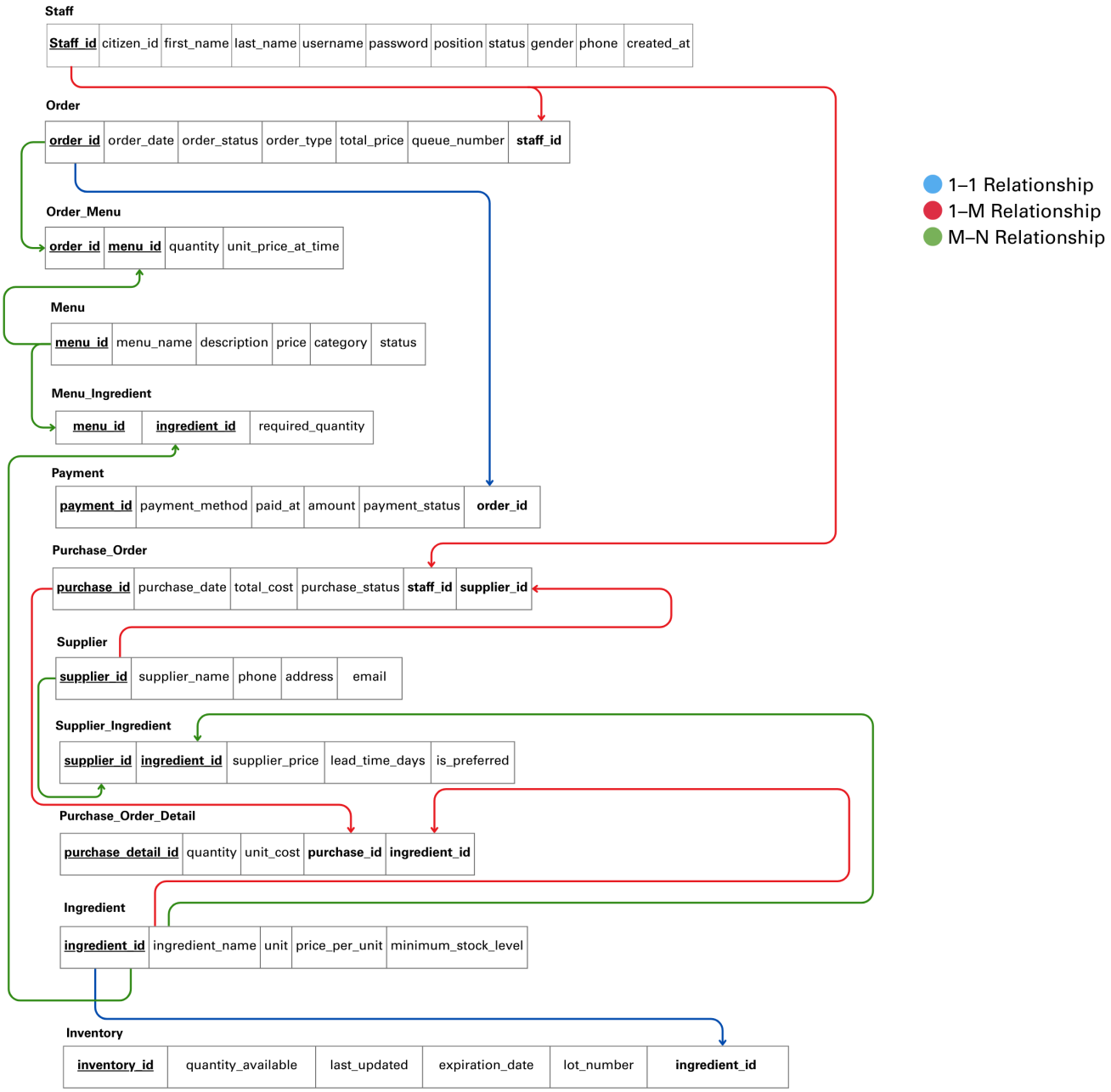
ER Diagram



https://www.canva.com/design/DAHCOBB3P74/tYWLtZa478CZyIx_Uc5Apw/view?utm_content=DAHCOBB3P74&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=h69a8955eee

Entity	Changes
Staff	**Add:** • username – Used for staff login. • password – Used for authentication.
Supplier	**Add:** • email – Used for supplier contact.
Supplier – Ingredient Relationship (M:N)	**Add Attributes:** • supplier_price – Price from supplier. • lead_time_days – Delivery time. • is_preferred – Preferred supplier indicator. **Note:** • These attributes belong to the **M:N relationship**, not a separate entity.
Ingredient	**Add:** • minimum_stock_level – Defines the minimum stock threshold for each ingredient.
Inventory	**Delete:** • minimum_stock_level – Removed from the design.
Order	**Add:** • order_type – Used to classify order types (Dine-in, Takeaway, Delivery).

Relational database schema



https://www.canva.com/design/DAHCOM4vx8Y/vUYHo6nBZtw3AlrELU0BwQ/view?utm_content=DAHCOM4vx8Y&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=h79c3f00fd3

Transformation from ER to Relational Model

Step 1: Transform Strong Entities

All strong entities in the ER diagram were converted into tables. Each entity became a relation and its identifier became the primary key.

The main tables include:

- **Staff**
  Primary key: staff_id
  Attributes include citizen_id, first_name, last_name, username, password, position, status, gender, phone, and created_at.
- **Supplier**
  Primary key: supplier_id
  Attributes include supplier_name, phone, and address.
- **Menu**
  Primary key: menu_id
  Attributes include menu_name, description, price, category, and status.
- **Ingredient**
  Primary key: ingredient_id
  Attributes include ingredient_name, unit, price_per_unit,and minimum_stock_level.
- **Order**
  Primary key: order_id
  Attributes include order_date, order_status, total_price, queue_number, and order_type.
- **Inventory**
  Primary key: inventory_id
  Attributes include quantity_available, last_updated, expiration_date, and lot_number.
- **Purchase_Order**
  Primary key: purchase_id
  Attributes include purchase_date, total_cost, and purchase_status.
- **Purchase_Order_Detail**
  Primary key: purchase_detail_id
  Attributes include quantity and unit_cost.
- **Payment**
  Primary key: payment_id
  Attributes include payment_method, paid_at, amount, and payment_status.

Step 2: Transform Weak Entities

There are **no weak entities** in this ER diagram.

Step 3: Transform 1:1 Relationships

For one-to-one relationships, a foreign key was added to one of the related tables.

- **Order – Payment**
  The attribute **order_id** was added to the Payment table as a foreign key.
  This ensures that each payment is linked to one order.
- **Ingredient – Inventory**
  The attribute **ingredient_id** was added to the Inventory table as a foreign key.
  This ensures that each ingredient has one inventory record.

Step 4: Transform 1:N Relationships

For one-to-many relationships, the primary key from the one side was added as a foreign key in the many side.
Examples include:

- **Staff → Order**
  Foreign key staff_id added to Order.
- **Staff → Purchase_Order**
  Foreign key staff_id added to Purchase_Order.
- **Supplier → Purchase_Order**
  Foreign key supplier_id added to Purchase_Order.
- **Purchase_Order → Purchase_Order_Detail**
  Foreign key purchase_id added to Purchase_Order_Detail.
- **Ingredient → Purchase_Order_Detail**
  Foreign key ingredient_id added to Purchase_Order_Detail.

These foreign keys represent the relationships in the relational model.

Step 5: Transform M:N Relationships

Many-to-many relationships were converted into separate tables.

- **Order_Menu**
  Primary key: order_id, menu_id
  Attributes: quantity, unit_price_at_time
  This table represents the relationship between Order and Menu.
- **Menu_Ingredient**
  Primary key: menu_id, ingredient
  Attribute: required_quantity
  This table represents the relationship between Menu and Ingredient.
- **Supplier_Ingredient**
  Primary key:supplier_id, ingredient_id
  Attributes:supplier_price, lead_time_days,is_preferred
  This table represents the relationship between Supplier and Ingredient.

Step 6: Transform Multivalued Attributes

There are **no multivalued attributes** in this ER diagram.

Step 7: Transform N-ary Relationships

There are **no N-ary relationships** in this ER diagram.

Step 8: Transform Subclass Relationships

There are **no subclass or superclass relationships** in this ER diagram.

Normalized relational database schema

Relation	Attribute	Key	Normalization	Relational Schema
Staff	staff_id {PK} citizen_id first_name last_name username password position status gender phone created_at	**PK:** staff_id	**Repeating Group (1NF):** There is no repeating group in the Staff relation. **Partial Dependency (2NF):** The primary key is a single attribute (staff_id), so partial dependency cannot occur. All non-key attributes fully depend on staff_id. **Dependency (3NF):** FD: staff_id → citizen_id, first_name, last_name, username, password, position, status, gender, phone, created_at There is no transitive dependency because all non-key attributes depend directly on staff_id, and no non-key attribute depends on another non-key attribute.	**R1) Staff(** staff_id {PK}, citizen_id, first_name, last_name, username, password, position status, gender, phone, created_at)
Order	order_id {PK} order_date order_status order_type total_price queue_number staff_id {FK}	**PK:** order_id **FK:** staff_id	**Repeating Group (1NF):** There is no repeating group in this relation. **Partial Dependency (2NF):** The primary key is a single attribute (order_id). Since there is no composite key, partial dependency cannot occur. All non-key attributes fully depend on order_id. **Dependency (3NF):** FD: order_id → order_date, order_status, order_type, total_price, queue_number, staff_id There is no transitive dependency because all non-key attributes depend directly on order_id. Staff details (e.g., staff_name) are not stored in this table, only staff_id as a foreign key.	**R2) Order(** order_id {PK}, order_date, order_status, order_type, total_price, queue_number, staff_id {FK})
Order_ Menu	order_id {PK} menu_id {PK} quantity unit_price_at_time	**PK:**Composite (order_id, menu_id) **FK:**order_id, menu_id	**Repeating Group (1NF):** There is no repeating group. Each row represents one menu item per order. **Partial Dependency (2NF):** The primary key is composite (order_id, menu_id). Both quantity and unit_price_at_time depend on the full composite key, not on order_id or menu_id alone. Therefore, there is no partial dependency. **Transitive Dependency(3NF):** FD: (order_id, menu_id) → quantity, unit_price_at_time There is no transitive dependency because non-key attributes do not depend on other non-key attributes. All non-key attributes depend directly on the full composite key. Therefore, the relation satisfies 3NF.	**R3) Order_Menu(** order_id {PK/FK, menu_id {PK/FK}, quantity, unit_price_at_time)

Menu	menu_id {PK} menu_name description price category status	**PK:** menu_id	**Repeating Group (1NF):** There is no repeating group. **Partial Dependency (2NF):** The primary key is a single attribute (menu_id). Since there is no composite key, partial dependency cannot occur. All non-key attributes fully depend on menu_id. **Transitive Dependency (3NF):** FD: menu_id → menu_name, description, price, category, status There is no transitive dependency because all non-key attributes depend directly on menu_id, and no non-key attribute depends on another non-key attribute. Therefore, the relation satisfies 3NF.	**R4) Menu**(menu_id {PK}, menu_name, description, price, category, status)
Menu_ Ingredient	menu_id {PK} ingredient_id {PK} required_quantity	PK (Composite):(menu_id, ingredient_id) FK:menu_id, ingredient_id	**Repeating Group (1NF):** There is no repeating group in the Menu_Ingredient relation. Each ingredient required for a menu is stored in a separate row. **Partial Dependency (2NF):** The primary key is a composite key (menu_id, ingredient_id). The attribute required_quantity depends on the full composite key, not on a single attribute. Therefore, no partial dependency exists. **Transitive Dependency (3NF):** FD: (menu_id, ingredient_id) → required_quantity There is no transitive dependency because required_quantity depends directly on the full composite key, and no non-key attribute depends on another non-key attribute.	**R5) Menu_Ingredient** (menu_id {PK/FK}, ingredient_id{PK/FK}, required_quantity)
Payment	payment_id {PK} payment_method paid_at amount payment_status order_id {FK}	PK: payment_id FK:order_id	**Repeating Group (1NF):** There is no repeating group in the Payment relation. All attributes store atomic values. **Partial Dependency (2NF):** The primary key is a single attribute (payment_id), so partial dependency cannot occur. All non-key attributes fully depend on payment_id. **Transitive Dependency (3NF):** FD: payment_id → payment_method, paid_at, amount, payment_status, order_id, staff_id There is no transitive dependency because all non-key attributes depend directly on payment_id, and no non-key attribute depends on another non-key attribute. Constraint: order_id is UNIQUE to enforce a 1:1 relationship between Order and Payment.	**R6) Payment** (payment_id {PK}, payment_method, paid_at, amount, payment_status, order_id {FK} (UNIQUE), staff_id{FK})

Supplier	supplier_id {PK} supplier_name phone address email	PK**:** supplier_id	**Repeating Group (1NF):** There is no repeating group in the Supplier relation. All attributes store atomic values. **Partial Dependency (2NF):** The primary key is a single attribute (supplier_id), so partial dependency cannot occur. All non-key attributes fully depend on supplier_id. **Transitive Dependency (3NF):** FD: supplier_id → supplier_name, phone, address, email There is no transitive dependency because all non-key attributes depend directly on supplier_id, and no non-key attribute depends on another non-key attribute.	**R7) Supplier**(supplier_id {PK}, supplier_name, phone, address, email)
Ingredient	ingredient_id {PK}, ingredient_name unit price_per_unit minimum_stock_level	PK: ingredient_id	**Repeating Group (1NF):** There is no repeating group in the Ingredient relation. All attributes contain atomic values. **Partial Dependency (2NF):** The primary key is a single attribute (ingredient_id), so partial dependency cannot occur. All non-key attributes fully depend on ingredient_id. **Transitive Dependency (3NF):** FD: ingredient_id → ingredient_name, unit, price_per_unit, minimum_stock_level There is no transitive dependency because all non-key attributes depend directly on ingredient_id, and no non-key attribute depends on another non-key attribute.	**R8) Ingredient** (ingredient_id {PK}, ingredient_name, unit, price_per_unit, minimum_stock_level)
Supplier_ Ingredient	supplier_id {PK} ingredient_id {PK} supplier_price lead_time_days is_preferred	PK (Composite):(supplier_id, ingredient_id) FK:supplier_id, ingredient_id	**Repeating Group (1NF):** There is no repeating group in the Supplier_Ingredient relation. Each supplier–ingredient pair is stored as one row. **Partial Dependency (2NF):** The primary key is a composite key (supplier_id, ingredient_id). All non-key attributes (supplier_price, lead_time_days, is_preferred) depend on the full composite key, not on a single attribute. Therefore, no partial dependency exists. **Transitive Dependency (3NF):** FD: (supplier_id, ingredient_id) → supplier_price, lead_time_days, is_preferred There is no transitive dependency because all non-key attributes depend directly on the full composite key, and no non-key attribute depends on another non-key attribute.	**R9) Supplier_Ingredient** (supplier_id {PK/FK}, ingredient_id {PK/FK}, supplier_price, lead_time_days, is_preferred)

Purchase_ Order	purchase_id {PK} purchase_date total_cost purchase_status staff_id {FK} supplier_id{FK}	PK: purchase_id FK:staff_id, supplier_id	**Repeating Group (1NF):** There is no repeating group in the Purchase_Order relation. Purchased items are stored in the Purchase_Order_Detail table. **Partial Dependency (2NF):** The primary key is a single attribute (purchase_id), so partial dependency cannot occur. All non-key attributes fully depend on purchase_id. **Transitive Dependency (3NF):** FD: purchase_id → purchase_date, total_cost, purchase_status, staff_id, supplier_id There is no transitive dependency because Purchase_Order does not store staff_name or supplier_name. All non-key attributes depend directly on purchase_id, and no non-key attribute depends on another non-key attribute.	**R10) Purchase_Order**(purchase_id {PK}, purchase_date, total_cost, purchase_status, staff_id {FK}, supplier_id {FK})
Purchase_ Order_Detail	purchase_detail_id {PK} quantity unit_cost purchase_id {FK} ingredient_id{FK}	PK:purchase_detail_id FK:purchase_id, ingredient_id	**Repeating Group (1NF):** There is no repeating group in the Purchase_Order_Detail relation. All attributes store atomic values. **Partial Dependency (2NF):** The primary key is a single attribute (purchase_detail_id), so partial dependency cannot occur. All non-key attributes fully depend on purchase_detail_id. **Transitive Dependency (3NF):** FD: purchase_detail_id → quantity, unit_cost, purchase_id, ingredient_id There is no transitive dependency because all non-key attributes depend directly on purchase_detail_id, and no non-key attribute depends on another non-key attribute.	**R11) Purchase_Order _Detail**(purchase_detail_id{PK}, quantity, unit_cost, purchase_id {FK}, ingredient_id {FK})
Inventory	inventory_id {PK} quantity_available last_updated expiration_date lot_number ingredient_id {FK}	PK: inventory_id FK:ingredient_id	**Repeating Group (1NF):** There is no repeating group. **Partial Dependency (2NF):** The primary key is a single attribute (inventory_id). Therefore, no partial dependency exists. **Dependency (3NF):** FD: inventory_id → lot_number lot_number → expiration_date, received_date Therefore: inventory_id → expiration_date, received_date This is a transitive dependency because expiration_date and received_date depend on lot_number, not directly on the primary key (inventory_id).	**R12) Inventory**(inventory_id {PK}, ingredient_id {FK}, quantity_available, last_updated) **R13) Ingredient_Batch**(batch_id {PK}, inventory_id {FK}, lot_number, quantity, original_quantity, expiration_date, received_date)

			To remove the transitive dependency, batch-related attributes were moved to a new table (Ingredient_Batch) with batch_id as the primary key. FD: batch_id → lot_number, quantity, original_quantity, expiration_date, received_date, inventory_id Additionally, **batch_id** (new primary key), **inventory_id** (foreign key), and batch-specific attributes such as **quantity** and **original_quantity** were introduced to support proper batch-level tracking and maintain referential integrity. As a result, all attributes now depend directly on their respective primary keys, and the relations satisfy Third Normal Form (3NF).	

Boyce–Codd Normal Form (BCNF)

Rule: A relation is in BCNF if for every functional dependency (X → Y), X is a superkey.

Implementation in the current schema:
All relations in the schema satisfy BCNF because every functional dependency is determined by a primary key or a full composite key.

For example, in the **Order** table:
order_id → order_date, order_status, total_price, staff_id
Since order_id is the primary key and a superkey, the relation satisfies BCNF.

Similarly, in the associative table **Order_Menu**:
(order_id, menu_id) → quantity, unit_price_at_time
The composite key (order_id, menu_id) is a superkey, and all non-key attributes fully depend on it. Therefore, this relation also satisfies BCNF.

No non-key attribute determines another non-key attribute in any relation. Hence, all determinants are superkeys, and the database schema satisfies Boyce–Codd Normal Form (BCNF).

Final Relational Schema

https://www.canva.com/design/DAHCOhrZJBg/9w6foSnF9FGcTr8VYIcpDA/view?utm_content=DAHCOhrZJBg&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=h70c4c9793f

This figure illustrates the final relational database schema of the restaurant management system. The diagram presents entity tables, primary keys (underlined), and foreign key relationships between tables.

The color-coded lines represent different relationship types:

- Blue indicates one-to-one (1–1) relationships.
- Red indicates one-to-many (1–M) relationships.
- Green indicates many-to-many (M–N) relationships resolved using associative tables.
- Yellow represents 3NF normalization (decomposition), such as the separation of Inventory and Ingredient_Batch to eliminate transitive dependency.

The schema ensures referential integrity and follows normalization principles up to Third Normal Form (3NF), and satisfies Boyce–Codd Normal Form (BCNF).